

Prueba Técnica – SII Laravel 12

Duración: 6–8 horas (máximo 1 día)
Modalidad: Individual
Entregables: Código corregido + `RESPUESTA.md` + (opcional) tests

Contexto

Se te entrega un **Sistema Integral de Información (SII)** simplificado para una universidad. El sistema gestiona alumnos, programas académicos, materias, inscripciones, pagos y adeudos.

El proyecto fue iniciado por un desarrollador junior y tiene **muchos errores, bugs y malas prácticas**. Además, **la base de datos contiene datos inconsistentes ingresados por usuarios/administradores** (errores humanos reales). Tu misión es identificar los problemas más críticos y corregirlos.

Nota importante: Este sistema utiliza **relaciones simbólicas** en la base de datos. No se utilizan *foreign keys* físicas ni `SoftDeletes`. Las relaciones y validaciones se manejan a nivel de aplicación.

Módulos existentes

Módulo	Descripción	Estado
Auth	Login, registro y roles (admin, sede, control escolar)	Funcional pero con fallos de seguridad
Dashboard	KPIs de alumnos, pagos, inscripciones	Lento, carga todo en memoria
Alumnos	CRUD de alumnos, inscripción y pago inicial	Sin paginación, sin validaciones, sin transacciones
Programas	Listado de programas y materias	Sin paginación, sin validación de tags JSON
Pagos	Registro de pagos por matrícula	No valida montos negativos, no verifica existencia de alumno
Reportes	Exporte CSV de deudas y generación async	Sin chunk, sin manejo de errores, sin caché
API	Endpoints para consultar alumno y pagos	Sin autenticación, sin rate limit, expone datos sensibles

Esquema de base de datos (simplificado)

```
users (id, name, email, password, nivel_id, sede_id, cargo)
sedes (id, nombre, clave, direccion, meta_alumnos, activa)
programas (id, nombre, objetivo, inscripcion, precio_materia, ...)
materias (id, nombre, clave, creditos, planescolar_id, ...)
programa_materia (id, programa_id, materia_id, orden)
alumnos (id, matricula, nombre_completo, apat, amat, curp, email, telefono, sede_id, ...)
inscripciones (id, alumno_id, programa_id, sede_id, estado, fecha_inscripcion, ...)
```

```
pagos (id, matricula, concepto, monto, fecha_pago, metodo, sede_id, estado, ...)
adeudos (id, alumno_id, nombre, monto, plazos, desde, dias, divisa_id)
reporte_deudas (id, nombre, filtros, resultado, estado)
```

Nota: Las relaciones entre tablas son **simbólicas** (no hay foreign keys físicas). Se manejan por convención de nombres (`*_id`) a nivel de aplicación.

Errores humanos en la base de datos (datos sucios)

El seeder `ErrorHumanosSeeder` insertó datos que simulan errores de captura por parte de usuarios. Debes identificar estos problemas y crear mecanismos para:

- **Prevenir** que entren (validaciones en aplicación, sanitización).
- **Detectar** los que ya existen (reportes, queries de auditoría).
- **Corregir** los afectados (scripts de limpieza, migraciones de corrección).

Ejemplos de errores humanos presentes:

- **Alumnos:** CURP inválida, email sin `@`, teléfono con letras, nombre vacío, espacios extra, matrícula duplicada, fecha de nacimiento futura, alumno de 2 años, estado no estándar (`en_proceso_baja`).
- **Sedes:** Nombre vacío (solo espacios), `meta_alumnos` negativa, sede inactiva con alumnos activos.
- **Programas:** Nombre vacío, precios negativos, tags en CSV en lugar de JSON, JSON malformado.
- **Usuarios:** Email inválido, duplicado, asignado a sede inactiva.
- **Inscripciones:** Programa inexistente, fecha de término anterior a inicio, múltiples inscripciones activas al mismo programa, sede nula.
- **Pagos:** Monto con comas (`1,500.00`), `monto = 0.00`, concepto vacío, fecha futura, alumno inexistente, estado no estándar (`pendiente_pago`), duplicado exacto, nota de 5000 caracteres (posible spam/ataque).
- **Adeudos:** Monto negativo, `plazos = 0`, alumno inexistente.

Tip: Crea un `DataQualityController` o un comando `php artisan data:audit` que liste todas las inconsistencias.

✓ Tareas sugeridas (elige las que más se ajusten al perfil)

1. Corrección crítica – Base de datos (30 min)

- Identificar por qué los campos de monto, matrícula y fecha están como `text` en lugar de tipos adecuados.
- Corregir las migraciones más importantes (o crear nuevas migraciones de corrección).
- Agregar índices en `alumnos(matricula)`, `alumnos(sede_id)`, `pagos(matricula)`, `inscripciones(alumno_id)`.
- **NO agregar foreign keys físicas** (las relaciones son simbólicas en este sistema).

2. Corrección crítica – Seguridad (45 min)

- Revisar `AlumnoController@index` y corregir el `whereRaw` que permite SQL injection.
- Corregir `NivelMiddleware` para usar `===` y manejar `null` / `nivel_id` inexistente.
- Cambiar la ruta de eliminación de alumno a `DELETE` y agregar confirmación.
- Proteger los endpoints de API con `auth:sanctum` o middleware de auth.
- Corregir CSRF en el formulario de creación de alumno.

3. Corrección crítica – Performance (45 min)

- Agregar paginación al listado de alumnos (`simplePaginate` o `paginate`).

- Corregir N+1 en `alumnos.index` usando `with('sede')` .
- Optimizar el `DashboardController` usando `DB::raw` o `selectRaw` para agregaciones.
- Agregar `chunk()` al exporte de CSV en `ReporteController` .
- Reemplazar el select de alumnos en el formulario de pagos por un buscador async o paginado.

4. Corrección crítica – Lógica de negocio (60 min)

- Validar que `matricula` sea única en `AlumnoController@store` (validación en aplicación, no constraint DB).
- Validar que `monto` sea positivo y numérico en `PagoController@store` .
- Validar que el alumno exista antes de registrar un pago.
- Envolver la creación de alumno + inscripción + pago en una `DB::transaction` .
- Validar que no se pueda eliminar un alumno con adeudos pendientes.
- Corregir el cálculo de comisión en `PagoController` (parseo de string a float).

5. Corrección crítica – Jobs y colas (30 min)

- Corregir `GenerarReporteDeudasJob` para manejar excepciones, usar `chunk()` , y verificar si `$reporte` existe.
- Agregar notificación o log si el job falla.
- (Opcional) Implementar un listener `JobFailed` .

6. Corrección crítica – Frontend y UX (30 min)

- Agregar un botón de confirmación JS antes de eliminar.
- Mejorar el select de sedes/alumnos con una búsqueda typeahead o paginación.
- (Opcional) Agregar un flash message de error si la validación falla.
- **NO requerir** `old()` (no es parte del flujo de este sistema).

7. Corrección crítica – Arquitectura (30 min)

- Extraer lógica de negocio de los controladores a Services o Actions (ej: `RegistrarAlumnoService`).
- Agregar `casts` en modelos para JSON, fechas y decimales.
- **NO requerir** `SoftDeletes` (no es parte del patrón de este sistema).
- **NO requerir** `FormRequest` (no es parte del patrón de este sistema; validar inline en controladores o en helpers).

8. Corrección crítica – Calidad de datos (30 min)

- Crear un comando o endpoint que audite y reporte todas las inconsistencias humanas (H1-H37).
- Implementar validaciones en aplicación para prevenir: CURP inválida, email malformado, teléfono con letras, fechas imposibles, montos negativos, estados no estándar.
- Sanitizar entradas: `trim()` en nombres y emails, normalizar mayúsculas/minúsculas, eliminar espacios.
- Crear un script de limpieza que corrija los datos existentes (ej: borrar pagos duplicados, unificar matrículas, corregir estados).
- Validar unicidad de `matricula` y `email` en aplicación (no agregar constraints de DB).

9. (Opcional) Tests (30 min)

- Escribir al menos 2 tests:
 - Un test que verifique que un alumno duplicado no se pueda registrar.
 - Un test que verifique que un pago negativo sea rechazado.
 - Corregir `phpunit.xml` si es necesario.
-

Estimación de tiempo total

Área	Tiempo estimado
DB + Modelos	30 min
Seguridad	45 min
Performance	45 min
Lógica de negocio	60 min
Jobs	30 min
Frontend	30 min
Arquitectura	30 min
Calidad de datos	30 min
Tests (opcional)	30 min
Documentación	30 min
Total	~6 horas

Criterios de evaluación

Ver `docs/EVALUACION.md` para la rúbrica completa.

Resumen:

- **Identificación de problemas:** ¿Encontró los bugs más críticos?
- **Calidad de soluciones:** ¿Usó las herramientas correctas de Laravel?
- **Seguridad:** ¿Corrigió SQL injection, roles, CSRF, auth?
- **Performance:** ¿Agregó paginación, eager loading, índices, chunk?
- **Arquitectura:** ¿Extrajo lógica, usó validaciones inline, transacciones?
- **Documentación:** ¿Explicó claramente sus decisiones?
- **Tests:** ¿Agregó tests que validen sus correcciones?

Entrega

1. **Crea un repositorio propio** (GitHub, GitLab, Bitbucket, etc.) con tu código corregido y tu `RESPUESTA.md`.
2. `RESPUESTA.md` en la raíz del proyecto.
3. (Opcional) **Capturas de pantalla** del antes/después del dashboard.

Envío

Al finalizar, envía el **enlace a tu repositorio** y tu `RESPUESTA.md` al correo:

`ltepepa@unisant.edu.mx`

Nota: Asegúrate de que el repositorio sea público o que nos compartas acceso de lectura.

¡Buena suerte! 🍀